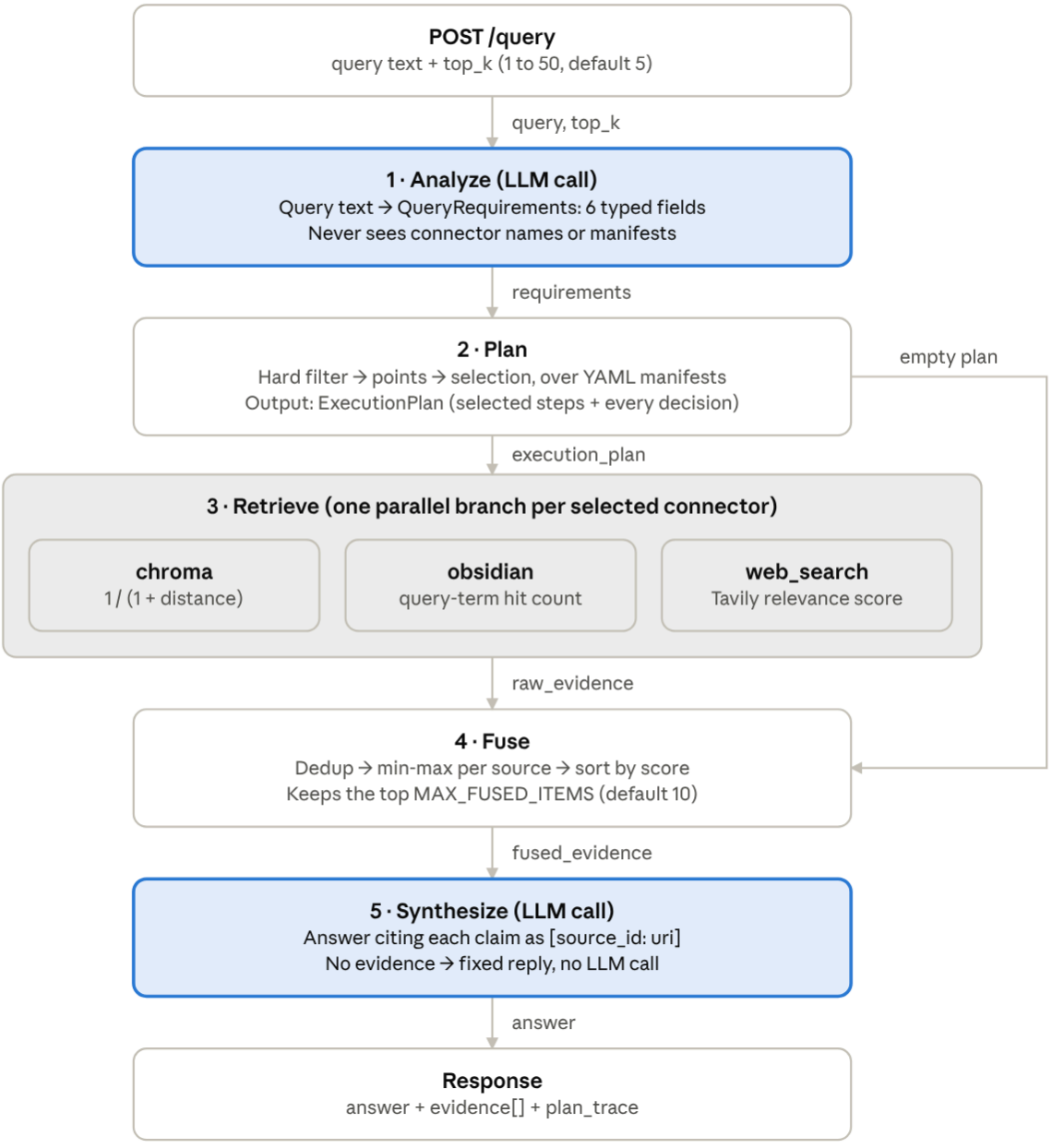
CapRetrieve: How a Query Flows End to End

At a glance

A query goes through five stages in one LangGraph graph. Only Analyze and Synthesize call an LLM; deciding which sources to query and how to rank their results is plain code you can read and test.

Two LLM calls wrap a deterministic routing core

Shaded steps call an LLM. Words on arrows are the GraphState keys handed on.



query flow · 5 stages, 2 LLM calls

If the plan selects no connector (for example, every source is privacy-blocked), retrieval is skipped and synthesis returns a fixed reply without calling the LLM.

1. Entry point: POST /query

A query enters as `POST /query` and is handed straight to a pre-built LangGraph graph; the route itself holds no logic (`api/routes/query.py`).

- **Request** (`api/schemas.py`): `{"query": str, "top_k": int}` . `top_k` defaults to 5 and is capped at 50 (`MAX_TOP_K`), because it is passed straight to paid Tavily calls and to Chroma.
- **Run**: the route calls `graph.ainvoke({"query", "top_k"})` . The graph was compiled once at startup by `bootstrap.py` → `graph/build.py` , with the connector registry and two LLM clients (analyzer, synthesis) injected.
- **Response**: `{answer, evidence[], plan_trace}` , where `answer` comes from synthesis, `evidence` is the fused list and `plan_trace` is built from the final graph state (section 7).

The graph is a fixed chain: `analyze → plan → retrieve_one × N (parallel) → fuse → synthesize` . Every stage reads and writes one shared `GraphState` dict (`graph/state.py`). The nodes in `graph/nodes/` are thin adapters; the real logic lives in `planner/` , `connectors/` , `fusion/` and `generation/` .

2. Analyze: query text → typed requirements

One LLM call turns the raw query into a `QueryRequirements` object. From here on the planner only sees these typed fields, never the query text (`planner/analyzer.py`).

The prompt (`planner/prompts/requirement_extraction.md`) asks for JSON with six fields:

Field	Values	What it drives later
`paradigm_hint`	dense_semantic, graph_traversal, keyword, unknown	+1.0 scoring bonus
`relational`	true / false	Hard filter: only graph_traversal connectors survive
`needs_freshness`	any, recent, live	+0.3 bonus; force-includes live connectors

Field	Values	What it drives later
`privacy_sensitivity`	none, personal, confidential	Hard filter: blocks connectors that leak the query
`expects_personal_knowledge`	true / false	+0.5 bonus; force-includes private_local connectors
`seed_entity`	string or null	Becomes a `seed_note` filter for graph traversal

- **Isolation:** the analyzer never sees connector names or manifests. It describes what the query needs, not where to send it. `tests/architecture/test_analyzer_isolation.py` enforces this.
- **Parsing:** `llm/structured.py` strips code fences, parses the JSON and validates it with Pydantic. On failure it replies to the model with the error and retries, up to 3 attempts in total, then raises `StructuredOutputError`.
- The call is blocking HTTP, so the node runs it in a thread (`asyncio.to_thread`).

3. Plan: pick which connectors to query

The planner matches the requirements against each connector's YAML manifest and outputs an `ExecutionPlan`: the connectors to query, plus a decision record for every candidate (`planner/planner.py`). There is no LLM in this stage.

The three connectors declare these manifests (`configs/connectors/*.yaml`):

Connector	paradigm	freshness	privacy_tier	query_leaks_externally
chroma	dense_semantic	index_time	private_local	false
obsidian	graph_traversal	live	private_local	false
web_search	keyword	live	public_external	true

The planner checks every manifest in three steps:

1. **Hard filter** (`planner/constraints.py`). A connector is eliminated, with a recorded reason, if either rule fires. Privacy is checked first, so the reason is deterministic.
   - `privacy_leak`: `privacy_sensitivity` is not `none` and the connector has `query_leaks_externally: true`.
   - `paradigm_mismatch`: `relational` is true and the connector isn't `graph_traversal`.

2. **Points** (`planner/scoring.py`). Survivors start at 0 and add fixed bonuses: +1.0 if the paradigm matches `paradigm_hint` , +0.5 if the query expects personal knowledge and the connector is `private_local` , +0.3 if the query needs freshness and the connector is `live` . The maximum is 1.8.
3. **Selection** (`planner/selection.py`).
   - Force-include every `private_local` connector when `expects_personal_knowledge` is true.
   - Force-include every `live` connector when `needs_freshness` is recent or live.
   - Keep any other survivor scoring at least 75% of the top score (`RELATIVE_THRESHOLD = 0.75`).
   - If the top score is 0, keep only the first survivor.
   - If nothing survives the filter, the plan is empty. That is a valid result: retrieval is skipped and the graph goes straight to fuse.

Selected connectors are sorted by score (ties by connector ID) into `ExecutionPlan.steps` . Excluded and unselected connectors stay in `ExecutionPlan.decisions` for the trace.

The planner receives only manifests, never connector objects or the registry. Architecture tests enforce this and also stop the planner reading the free-text `scope` field.

4. Retrieve: query the selected connectors in parallel

Each selected connector gets its own parallel `retrieve_one` branch (one LangGraph `Send` per plan step), and their evidence lists are concatenated into `raw_evidence` (`graph/nodes/retrieve.py`).

- **Request:** every branch gets the same `RetrievalRequest` : the original query text, `top_k` and optional `filters` .
- **Filters:** if the query is `relational` and has a `seed_entity` , every branch gets `{"seed_note": <entity>, "depth": 1}` . Only Obsidian uses it; the others ignore it.
- **Empty plan:** the router skips retrieval and goes straight to fuse.
- **Failures:** a connector that raises adds an entry to `retrieval_errors` instead of evidence. The other branches still reach fuse, so one bad source doesn't fail the query.

Each connector returns `Evidence` items (`source_id` , `content` , `score` , `uri` , `retrieved_at`), with its own kind of score:

Connector	How it searches	Raw score
chroma	Embedding similarity over the local collection, `min(top_k, count)` results	`1 / (1 + distance)`, so higher is better
obsidian, no seed	Counts whole-word query-term hits in title, content and tags, minus stopwords	Hit count (1, 2, 3, …)
obsidian, with seed	Finds the seed note case-insensitively, then follows wikilinks to depth 1. Falls back to keyword search if no note matches.	1.0 for every linked note
web_search	Tavily search with `max_results = top_k`	Tavily's relevance score

Obsidian reloads the vault whenever a file's modification time changes, which is what justifies its `freshness: live` manifest.

What `uri` is

`uri` is an optional field on each `Evidence` item that says where the result came from: a file path or a web URL (`contracts/result.py`).

Connector	`uri` value
chroma	The source document's path saved at ingest (`source_path`), or the Chroma document ID if that's missing
obsidian	The note's file path
web_search	The result's URL from Tavily

Two later stages use it. Fuse uses `(source_id, uri)` as the key for removing duplicates (section 5). Synthesize shows it to the LLM, so the answer can cite each claim as `[source_id: uri]`, for example `[obsidian: Standing desk decision.md]`, and you can trace any sentence back to the exact note, document or page.

5. Fuse: merge everything into one ranked list

Fusion turns all the raw evidence into one list ranked best-first and capped at `MAX_FUSED_ITEMS` (default 10). It has no LLM call (`fusion/merge.py` , enforced by `tests/architecture/test_fusion_has_no_llm.py`).

1. **Dedup** (`fusion/dedup.py`): if one source returned the same `uri` twice, keep the higher-scoring copy. Items without a `uri` always pass through. Results from different sources are never merged, so each keeps its provenance.
2. **Normalize** (`fusion/normalize.py`): min-max scale scores to 0–1 separately for each `source_id` . Each source's best item becomes 1.0 and its worst 0.0. If every item from a source has the same score, all become 1.0.
3. **Sort and cap:** sort by normalized score, highest first, and keep the top `max_fused_items` . Ties keep their arrival order.

How dedup picks the winner

The `uri` itself isn't scored. Dedup compares the scores that connectors already put on each item, and keeps one item per `(source_id, uri)` pair.

1. Group items by `(source_id, uri)` . Items with no `uri` skip this step and are always kept.
2. In each group, keep the item with the highest raw score: the connector's own score, before normalization.
3. On a tie, keep the first one seen, because the comparison is a strict `>` .
4. Put the winner where that `uri` first appeared, and drop the other copies.

For example, if Chroma returned `notes.md` twice with scores 0.4 and 0.7, only the 0.7 copy stays.

With today's connectors this almost never fires. Chroma stores one document per file (`ids = p.stem` , no chunking). Obsidian scores each note once, and traversal tracks a `visited` set. Web search would need Tavily to return the same URL twice in one response. Dedup never compares across sources, so Obsidian and web results on the same topic are both kept. The step would start to matter if Chroma split files into chunks that share one `source_path` .

Because scaling is per source, every source's best hit ties at 1.0, so the top of the list is shared roughly evenly across sources. Normalized scores rank items within a source, but they aren't a real cross-source relevance signal. Planner scores are not used here.

6. Synthesize: write the cited answer

A second LLM call writes the final answer, using only the fused evidence and citing each claim as `[source_id: uri]` (`generation/synthesize.py`).

- **Prompt** (`generation/prompts/synthesis.md`): the question plus a numbered evidence block. Each entry is `[i] source_id=… uri=…` followed by the content. The model is told not to use outside knowledge and to say plainly when the evidence is insufficient.
- **No evidence, no call:** if `fused_evidence` is empty, the answer is the fixed string *"No permitted source returned evidence for this query."* and no LLM call is made. This stops a privacy-blocked query (empty plan) from reaching the external LLM provider.
- The answer is plain text; it isn't parsed or validated. The API returns it as `answer` , next to the fused `evidence` list.
- Like the analyzer, the call runs in a thread. The client is injected separately, so synthesis could use a different model from analysis.

7. Trace: see why it routed that way

Every run produces a `PlanTrace` , built after the graph finishes from its final state (`trace/recorder.py`). It is the best way to see each decision for a real query.

Trace field	What it shows
`query`	The original text
`requirements`	The analyzer's six typed fields
`execution_plan.decisions`	Every connector: admitted with its score, or excluded with `constraint` and `detail`
`execution_plan.steps`	The selected connectors, in score order
`execution`	Per selected connector: `evidence_count` (0 is kept) and `error` if it raised
`fused_evidence`	The final ranked list handed to synthesis
`answer`	The synthesized answer (empty when synthesis didn't run)

Three ways to get one:

- `POST /query` : returns it as `plan_trace` next to the answer.

- `POST /trace`: runs the graph without synthesis, so routing can be checked cheaply. Only the analyzer LLM call runs; retrieval still runs for real.
- `GET /trace/view?query=…&token=…`: runs the full graph and renders the trace as an HTML page. It needs `TRACE_VIEW_TOKEN` to be set and passed as `token`; otherwise it returns 403.

`scripts/show_trace.py` exists but is empty (0 bytes), as are the other files in `scripts/`.

8. Worked example: one query, every stage

Query (scenario S2 in `fixtures/scenarios.yaml`): *"What did I conclude about switching to a standing desk, and has anything changed since?"* With the analyzer output below, all three connectors are queried. Two of them get in through force-include rules, not their scores.

The analyzer output and raw retrieval scores are illustrative, since they depend on the LLM and your data. The plan and fused scores come from running the repo's `planner.plan()` and `fusion.merge.fuse()` on those inputs.

1. Analyze. A plausible `QueryRequirements`: `paradigm_hint=dense_semantic`, `relational=false`, `needs_freshness=recent`, `privacy_sensitivity=none`, `expects_personal_knowledge=true`, `seed_entity=null`.

2. Plan. No hard rule fires: privacy is `none` and the query isn't relational. Scores and selection:

Connector	Bonuses	Score	Selected because
chroma	+1.0 paradigm, +0.5 personal/local	1.5	Top score
obsidian	+0.5 personal/local, +0.3 freshness/live	0.8	Force-included: personal knowledge and `private_local` (0.8 is below the 1.125 threshold)
web_search	+0.3 freshness/live	0.3	Force-included: freshness and `live`

Plan steps: `chroma (1.5) → obsidian (0.8) → web_search (0.3)`.

3. Retrieve. Three parallel branches with `top_k=5` and no filters. Obsidian runs keyword search because there's no seed. Say they return:

- chroma: `sit-stand-review.pdf` 0.62, `sedentary-risk.pdf` 0.55, `posture.pdf` 0.41
- obsidian: `Standing desk decision.md` 3 hits, `Home office.md` 1 hit
- web_search: `…/standing-desk-study-2026` 0.81, `…/desk-guide` 0.77

4. Fuse. Per-source min-max, then sort:

Rank	Source	Item	Raw	Normalized
1	chroma	sit-stand-review.pdf	0.62	1.00
2	obsidian	Standing desk decision.md	3	1.00
3	web_search	standing-desk-study-2026	0.81	1.00
4	chroma	sedentary-risk.pdf	0.55	0.67
5	chroma	posture.pdf	0.41	0.00
6	obsidian	Home office.md	1	0.00
7	web_search	desk-guide	0.77	0.00

Each source's top hit ties at 1.0, and ties keep arrival order. The web result at 0.77 drops to 0.0 only because it was the weaker of two web results.

5. Synthesize. All 7 items (under the cap of 10) go into the prompt as `[1] source_id=chroma uri=sit-stand-review.pdf …` through `[7]`. The answer cites claims like `[obsidian: Standing desk decision.md]`.

6. Trace. `plan_trace.execution` reports chroma 3, obsidian 2 and web_search 2 items, with no errors.

In the fixtures, S2 is marked `deferred` to Phase 5. It is meant to need sequential composition (local conclusion first, then a web gap-fill), which today's parallel fan-out doesn't do.

9. Knobs and gotchas

Routing changes through manifests and a few code constants. `configs/weights.yaml` and `configs/settings.yaml` exist but are both empty.

Knob	Where	Default	Affects
`top_k`	Request body, or `DEFAULT_TOP_K` env	5 (max 50)	Results per connector
`MAX_FUSED_ITEMS`	Env	10	Items handed to synthesis

Knob	Where	Default	Affects
Connector manifests	`configs/connectors/*.yaml`	See section 3	Hard filter, bonuses, force-includes, with no code change
Bonus points	`planner/scoring.py` constants	1.0 / 0.5 / 0.3	Connector scores
`RELATIVE_THRESHOLD`	`planner/selection.py`	0.75	How close to the top score a connector must be
`DEFAULT_TRAVERSAL_DEPTH`	`graph/nodes/retrieve.py`	1	Wikilink hops for seeded Obsidian queries
`ANALYZER_MODEL`, `SYNTHESIS_MODEL`	Env	`anthropic/claude-sonnet-5`	The two LLM calls (via OpenRouter)

Things that surprise people:

- **The analyzer decides almost everything.** For S1 ("search the web for advice on my divorce settlement…"), if the LLM returns `paradigm_hint=keyword` but misses `expects_personal_knowledge`, both local connectors score 0. Only `chroma` runs, because it is first in registry order. With that flag set, both run. (Checked with the real planner.)
- **Obsidian counts as live.** Its manifest says `freshness: live`, so every query needing recent or live data force-includes Obsidian as well as web search.
- **Fused scores aren't comparable across sources.** Each source's best hit becomes 1.0, and a source's weaker result can drop to 0.0 next to a stronger one from elsewhere (section 8).
- **An analyzer failure fails the request.** After 3 bad JSON attempts `StructuredOutputError` is raised, and nothing in the graph or API catches it. Connector failures, by contrast, are absorbed into `retrieval_errors`.
- **Chroma blocks the event loop.** Its `collection.query` runs directly inside the async `retrieve()`, unlike Tavily, which uses `asyncio.to_thread`. A slow Chroma query stalls the other parallel branches.
- **`deadline_ms` is never set.** `RetrievalRequest` has the field, but the graph doesn't fill it, so web search has no timeout from the pipeline.